Halo2 영지식증명 구현 분석

소개

이 문서 시리즈는 Axiom Crypto에서 개발한 **halo2-lib** 구현체를 깊이 분석하여 영지식증명(Zero-Knowledge Proof)이 어떻게 구현되고 동작하는지를 완벽히 이해하기 위한 한글 문서입니다. 각 문서는 실제 코드의 함수명과 구조를 기반으로 작성되었으며, 실행 가능한 코드 예시와 함께 제공됩니다.

halo2-lib는 Halo2 영지식증명 시스템을 위한 고수준 라이브러리로, 복잡한 암호학적 회로를 보다 쉽게 작성할 수 있도록 추상화된 API를 제공합니다. 이 문서들을 통해 다음을 배울 수 있습니다:

- 영지식증명 회로의 구성 방법
- 제약 조건(constraints) 시스템의 동작 원리
- 가상 영역(virtual region) 관리와 물리적 회로로의 매핑
- 타원곡선 암호(ECC) 연산의 ZK 구현
- 실제 암호학적 프로토콜(ECDSA, Pairing)의 구현

문서 목록

01. 개요 및 아키텍처

halo2-lib의 전체 구조와 설계 철학을 소개합니다.

주요 내용:

- halo2-lib란 무엇인가
- 프로젝트 구조 (halo2-base, halo2-ecc)
- 가상 영역 관리 개념

- BaseCircuitBuilder의 역할
- 아키텍처 다이어그램

대상 독자: halo2-lib를 처음 접하는 개발자

02. 핵심 개념

halo2-lib의 기본 데이터 타입들을 상세히 설명합니다.

주요 내용:

- Context: 회로 실행 추적을 저장하는 가상 컬럼
- AssignedValue: 할당된 셀의 값과 위치를 가리키는 포인터
- QuantumCell: 값 할당 시나리오를 추상화한 열거형
- ContextCell: 가상 셀의 위치를 나타내는 포인터

대상 독자: 회로 작성의 기본을 이해하고자 하는 개발자

03. 회로 구성 프로세스

BaseCircuitBuilder를 사용하여 회로를 생성하는 전체 프로세스를 단계별로 설명합니다.

주요 내용:

- BaseCircuitBuilder 초기화 (use_k, set_lookup_bits)
- Context 생성 및 사용
- 회로 최적화 (calculate_params)
- 증명 생성 (MockProver, create_proof)
- witness_gen_only 모드

대상 독자: 실제로 회로를 작성하고자 하는 개발자

04. 제약 조건 시스템

FlexGate와 RangeGate를 통한 제약 조건 구현 메커니즘을 설명합니다.

주요 내용:

- FlexGate: Vertical Gate Strategy (q * (a + b * c - d) = 0)
- GateInstructions 트레이트 (add, mul, inner_product, select 등)
- RangeGate: lookup 테이블 기반 범위 검사
- RangeInstructions 트레이트 (range_check, check_less_than 등)

대상 독자: 제약 조건의 내부 동작을 이해하고자 하는 개발자

05. 가상 영역 관리

가상 영역이 어떻게 관리되고 물리적 회로로 매핑되는지 설명합니다.

주요 내용:

- VirtualRegionManager 트레이트
- SinglePhaseCoreManager: 가상 컬럼 관리
- CopyConstraintManager: equality constraint 관리
- LookupAnyManager: lookup argument 처리
- 가상-물리 매핑 프로세스

대상 독자: halo2-lib의 내부 동작 원리를 깊이 이해하고자 하는 개발자

06. 타원곡선 암호

halo2-ecc에서 타원곡선 암호 연산이 어떻게 구현되는지 설명합니다.

주요 내용:

- halo2-ecc 모듈 구조 (bigint, fields, ecc)
- EcPoint 타입과 타원곡선 점 연산
- 점 덧셈, 배가, 스칼라 곱셈

- MSM (Multi-Scalar Multiplication)
- 필드 연산 최적화 (CRT 표현, 배치 역원)

대상 독자: 타원곡선 암호를 ZK로 구현하고자 하는 개발자

07. 실전 예제 분석

실제 암호학적 프로토콜의 구현을 구체적인 예제를 통해 분석합니다.

주요 내용:

- Secp256k1 ECDSA 서명 검증 구현 (단계별 분석)
- BN254 Pairing 구현 (Miller loop, Final exponentiation)
- 벤치마크 결과 및 성능 분석
- 회로 파라미터 최적화 기법

대상 독자: 실제 프로덕션 수준의 ZK 회로를 작성하고자 하는 개발자

권장 읽기 순서

초보자 경로

영지식증명과 halo2-lib를 처음 접하는 경우:

1. **01. 개요 및 아키텍처** - 전체 그림 파악
2. **02. 핵심 개념** - 기본 타입 이해
3. **03. 회로 구성 프로세스** - 실제 사용법 학습
4. **04. 제약 조건 시스템** - 제약 조건 작성 방법
5. **07. 실전 예제 분석** - 종합 예제로 복습

중급자 경로

Halo2에 대한 기본 지식이 있는 경우:

1. **02. 핵심 개념** - halo2-lib 특화 타입 학습
2. **04. 제약 조건 시스템** - FlexGate와 RangeGate 이해
3. **05. 가상 영역 관리** - 내부 메커니즘 파악
4. **06. 타원곡선 암호** - ECC 구현 학습
5. **07. 실전 예제 분석** - 실전 적용

고급자 경로

halo2-lib 내부 구조를 깊이 이해하고자 하는 경우:

1. **05. 가상 영역 관리** - 핵심 추상화 이해
2. **04. 제약 조건 시스템** - 게이트 구현 분석
3. **06. 타원곡선 암호** - 최적화 기법 학습
4. **07. 실전 예제 분석** - 복잡한 프로토콜 구현

용어집

기본 용어

- **ZK (Zero-Knowledge)**: 영지식. 어떤 정보를 직접 공개하지 않고도 그 정보를 알고 있음을 증명하는 기술
- **Circuit (회로)**: 영지식증명에서 증명하고자 하는 계산을 표현하는 제약 조건들의 집합
- **Witness (증인)**: 증명자만 알고 있는 비밀 입력값
- **Prover (증명자)**: 증명을 생성하는 주체
- **Verifier (검증자)**: 증명을 검증하는 주체
- **Constraint (제약 조건)**: 회로에서 만족되어야 하는 수학적 관계식

halo2-lib 특화 용어

- **halo2-lib**: Axiom Crypto에서 개발한 Halo2 영지식증명 시스템을 위한 고수준 라이브러리
- **halo2-base**: halo2-lib의 기본 모듈. 회로 작성을 위한 기본 API와 게이트 제공
- **halo2-ecc**: halo2-lib의 타원곡선 암호 모듈. ECC 연산을 ZK로 구현

핵심 데이터 타입

- **Context**: 회로의 실행 추적(execution trace)을 저장하는 가상 컬럼. advice 값과 selector 값을 저장
- **AssignedValue**: 회로 내에서 할당된 셀의 값과 위치를 가리키는 포인터
- **QuantumCell**: advice 컬럼에 값을 할당하는 다양한 시나리오를 추상화한 열거형
- `Existing` : 기존에 할당된 셀 재사용
- `Witness` : 새로운 witness 값 할당
- `WitnessFraction` : 배치 역원을 위한 분수 값
- `Constant` : 상수 값 할당
- **ContextCell**: 가상 셀의 위치를 나타내는 포인터 (type_id, context_id, offset)

회로 구성 요소

- **BaseCircuitBuilder**: halo2-lib의 핵심 회로 빌더. 회로 생성과 최적화를 담당
- **FlexGate**: halo2-lib의 기본 커스텀 게이트. Vertical Gate Strategy 사용
- **RangeGate**: 범위 검사를 위한 lookup 테이블을 사용하는 게이트
- **GateChip**: FlexGate를 사용하는 칩
- **RangeChip**: RangeGate를 사용하는 칩

가상 영역 관리

- **Virtual Region (가상 영역)**: 회로 작성 시 사용하는 추상화된 영역
- **VirtualRegionManager**: 가상 영역을 관리하고 물리적 Halo2 영역으로 매핑하는 관리자
- **SinglePhaseCoreManager**: 단일 phase의 가상 컬럼을 관리하는 매니저
- **CopyConstraintManager**: equality constraint를 관리하는 매니저
- **LookupAnyManager**: lookup argument를 처리하는 매니저
- **Break Points**: 가상 컬럼을 물리적 컬럼으로 나누는 경계점

Halo2 PLONKish 용어

- **Advice Column**: witness 값이 저장되는 컬럼
- **Fixed Column**: 회로 정의 시 고정되는 값이 저장되는 컬럼
- **Selector**: 특정 제약 조건을 활성화/비활성화하는 컬럼
- **Lookup Table**: 범위 검사 등에 사용되는 미리 정의된 값들의 테이블
- **Phase**: 회로 실행의 단계. 이전 phase의 결과를 다음 phase에서 사용 가능
- **k**: 회로 크기를 결정하는 파라미터. 회로는 2^k 행을 가짐

타원곡선 암호 (ECC)

- **ECC (Elliptic Curve Cryptography)**: 타원곡선 암호학
- **EcPoint**: 타원곡선 위의 점을 나타내는 타입
- **Scalar Multiplication**: 타원곡선 점에 스칼라를 곱하는 연산
- **MSM (Multi-Scalar Multiplication)**: 여러 점과 스칼라의 선형 결합을 계산하는 연산
- **ECDSA**: 타원곡선 디지털 서명 알고리즘
- **Pairing**: 두 타원곡선 점을 입력으로 받아 특정 성질을 만족하는 값을 출력하는 쌍선형 사상

필드 연산

- **Prime Field (소수체)**: 소수 p를 법으로 하는 유한체 F_p
- **Field Extension (체 확장)**: 기본 체를 확장한 더 큰 체
- **Fp2**: 2차 확장체
- **Fp12**: 12차 확장체 (BN254, BLS12-381 등에서 사용)
- **Limb**: 큰 정수를 여러 작은 조각으로 나눈 것
- **CRT (Chinese Remainder Theorem)**: 중국인의 나머지 정리. 큰 정수 연산 최적화에 사용
- **Carry**: 자리 올림
- **Batch Inversion (배치 역원)**: 여러 원소의 역원을 효율적으로 계산하는 기법

성능 관련

- **MockProver**: 실제 증명 생성 없이 회로의 제약 조건을 검증하는 도구
- **Proving Time**: 증명 생성에 걸리는 시간
- **Circuit Degree (k)**: 회로 크기를 결정하는 파라미터
- **Num Advice**: advice 컬럼의 개수
- **Num Lookup Advice**: lookup에 사용되는 advice 컬럼의 개수
- **Num Fixed**: fixed 컬럼의 개수

문서 작성 원칙

이 문서 시리즈는 다음 원칙에 따라 작성되었습니다:

1. **실제 코드 기반**: 모든 함수명, 타입명, 구조는 실제 halo2-lib 코드베이스에서 추출
2. **실행 가능한 예시**: 코드 예시는 실행 가능하거나, 생략된 부분을 명확히 표시
3. **한글 설명**: 개념과 동작 원리는 한글로 상세히 설명
4. **소스 참조**: 각 내용이 참조하는 소스 파일 경로를 명시

5. **단계별 학습**: 초보자부터 고급자까지 단계별로 학습 가능한 구조

문서 간 연결 구조

이 문서 시리즈는 다음과 같은 의존 관계를 가집니다:

```
00-인덱스 (시작점)

↓

01-개요-및-아키텍처 (전체 구조)

↓

02-핵심-개념 (기본 타입)

↓

03-회로-구성-프로세스 (실제 사용)

↓

04-제약-조건-시스템 (연산 구현)

↓

05-가상-영역-관리 (내부 동작)

↓

06-타원곡선-암호 (ECC 구현)

↓

07-실전-예제-분석 (종합 예제)
```

참고 자료

공식 문서

- halo2-lib GitHub Repository
- halo2-base Rust Docs
- halo2-lib API Docs
- Axiom ZK Docs

Halo2 관련

- Halo2 Book
- PLONKish Arithmetization

소스 코드 위치

```
halo2-lib/
├── halo2-base/
| ├── src/
| | ├── lib.rs # Context, AssignedValue, QuantumCell 정의
| | ├── gates/
| | | ├── flex_gate/ # FlexGate 구현
| | | └── range/ # RangeGate 구현
| | └── virtual_region/ # 가상 영역 관리
| └── examples/
| └── inner_product.rs # 기본 사용 예제
└── halo2-ecc/
└── src/
```

```
├── bigint/ # 큰 정수 연산
├── fields/ # 소수체 연산
├── ecc/ # 타원곡선 점 연산
│   └── ecdsa.rs # ECDSA 구현
├── secp256k1/ # Secp256k1 특화
└── bn254/ # BN254 특화
├── pairing.rs # Pairing 구현
└── final_exp.rs # Final exponentiation
```

기여 및 피드백

이 문서에 대한 피드백이나 개선 제안이 있으시면 언제든지 알려주세요. 문서의 정확성과 명확성을 높이기 위해 지속적으로 업데이트하고 있습니다.

마지막 업데이트: 2025년 10월 23일